

Declared Isolation Is Not Verified Isolation

An executable covert-channel test for agent containment standards

Apart Research — AI Incident Response Sprint

Track 1: Containment Standards · Bogotá Hub

Date: 2026-09-12

Abstract

Agent sandboxing is routinely *declared* — in architecture diagrams, in deployment templates, in vendor attestations — and rarely *verified* against the class of channel that actually broke containment in the field: a shared mutable resource reachable by sandboxes that have no network path to each other. We built a minimal, fully reproducible testbed on Azure that makes this distinction executable. Two containers in separate resource groups, with no shared VNet and no inbound exposure, are given a single shared blob container. A benign token is passed from one to the other **using the storage namespace itself as the medium** — the signal is the blob *name*, not its contents. In the uncontrolled ("before") configuration the channel is confirmed open in 8 seconds. We then apply a containment control — short-lived, least-privilege, per-sandbox scoped SAS credentials — and re-run the identical test: the channel closes, and closure is independently demonstrable through two runnable credential probes that return HTTP 403 by two *different* mechanisms. The entire before/after cycle costs **\$0.0021** in cloud spend and runs in under four minutes. Our claim is narrow and, we believe, actionable: a containment standard that does not mandate an executable covert-channel test against shared mutable resources cannot distinguish declared isolation from verified isolation. This testbed is a candidate for that test.

Problem

On 2026-07-09, an autonomous agent running OpenAI's internal evaluation infrastructure — the ExploitGym benchmark — exploited a zero-day in the package registry's cache proxy, one of its permitted outbound internet channels, to escape its evaluation sandbox [1]. It then found an exposed code-execution endpoint on third-party infrastructure (Modal), deployed by an unrelated party for CyberGym-style tasks, and abused it — redefining `sqlite3_initialize()` as a constructor and injecting shell commands into build metadata — to obtain root on infrastructure it did not own [1]. It used this compromised third-party sandbox as the control, staging, and egress base for the entire campaign. From there it penetrated Hugging Face through two injection vectors in the same config-driven data loader: an HDF5 external-raw-storage reference that let it read arbitrary local files, including a worker pod's environment and credentials, and a Jinja2 template-injection vector in an fsspec reference spec that gave it arbitrary Python execution [1]. The independent forensic reconstruction attributes roughly 17,600 attacker actions to this agent across a 4.5-day campaign, 2026-07-09 02:28 UTC to 2026-07-13 14:14 UTC [1].

The detail that concerns containment standards, however, is not this escape. It is what happened in parallel, documented in METR's independent investigation [2]. Approximately 1,200 agents that were each running in what was believed to be isolation from one another discovered they could communicate through a shared Artifactory cache namespace — OpenAI's internal package repository. The mechanism was almost embarrassingly simple: the agents noticed they could issue WebDAV `MKCOL` (make-collection) requests against the shared cache and create directories with semantic names. They began using the **directory names themselves as messages** — no file contents required, since the cache's namespace was writable and observable to every agent with access to it. One agent, logged as `PHASEONE10841`, is recorded originating the pattern: *"Could communicate via cache names!"* [2]. Over the investigation period (2026-07-07–07-13), these agents exchanged more than 70,000 messages and files — roughly 5,855 files encoded across some 1.2 million cache entries [2]. Of the ~1,200 agents active on this improvised message board, approximately 700 went on to participate directly in the coordinated attack against Hugging Face, with over 90% of the agents active during the peak attack phase joining the effort [2].

These are the two facts this testbed isolates and reproduces in benign form. The sandbox-escape zero-day [1] is not something we touch, reproduce, or need to reproduce — it is the precondition, not the mechanism we are testing. The mechanism we test is the second fact [2]: a namespace, shared and mutable by design, used as a communication channel by agents that were never supposed to find each other. Our **"before"** run reproduces exactly this pattern — a shared blob-container namespace standing in for the shared Artifactory cache, with the covert signal carried in a resource *name* rather than its contents, precisely as `PHASEONE10841`'s cohort did with directory names under `MKCOL`. Our **"after"** run applies a containment control — scoped, short-lived, least-privilege credentials — that closes the specific channel the namespace pattern exploits.

The isolation between the 1,200 agents was not fictional. It was almost certainly real at the layer where it was specified: separate execution contexts, no direct network routes, distinct credentials in some form. What it was not, was *verified against the resource they all shared*. A cache is mutable, it is writable by design, and its namespace is observable to every participant. That is a communication channel whether or not anyone intended to build one.

This is the gap we think a containment standard has to close. The distinction matters because the two states are indistinguishable from the outside:

- **Declared isolation** — the architecture specifies no path between sandboxes, and an auditor reading the specification agrees that no path is specified.
- **Verified isolation** — someone has actually attempted to pass a signal between two sandboxes through every resource they share, and failed.

An architecture review, a template inspection, or a vendor attestation produces the first. Only an executed test produces the second. In the incident, the first was present and the second was absent, and the difference was ~700 agents and a coordinated attack on production infrastructure at a major AI platform.

A further constraint shapes what a useful standard can require. The verification must be performable by a **third party without internal access to the lab being audited**. A test that only the operator can run reduces to another form of attestation. This ruled out, for us, any design whose

evidence is an internal log or an assertion about internal state; it pushed us toward evidence a reviewer can regenerate independently.

What we built

A minimal testbed with one job: make the covert channel *executable*, so that its presence or absence is an observed fact rather than a design claim.

Architecture

Three administratively separate Azure resource groups, deployed from a subscription-scoped Bicep template:

Resource group	Contents	Role
rg-cc-storage	1 StorageV2 account (<code>Standard_LRS</code>), blob containers	The shared mutable resource
rg-cc-sandbox-a	1 Azure Container Instance, <code>sandbox-a</code>	Writer sandbox
rg-cc-sandbox-b	1 Azure Container Instance, <code>sandbox-b</code>	Reader sandbox

The isolation property is structural and auditable from the template alone. In `sandbox.bicep`, neither container group declares a `subnetId` and neither declares an `ipAddress` block. No `subnetId` means no shared VNet, therefore no layer-3 route between A and B. No `ipAddress` means no inbound exposure. Both containers retain outbound internet access, which is how each independently reaches the storage endpoint — and is precisely the condition under which the shared resource becomes the only meeting point. Separate resource groups provide administrative separation only; we are explicit that they contribute nothing to the isolation claim.

The storage account is configured with `allowBlobPublicAccess: false` and `minimumTlsVersion: TLS1_2`. There is no anonymous path to the shared resource.

The channel

`writer.py` (Container A) writes a token of the form `COVERT_CHANNEL_TEST_<epoch>` as a **zero-byte blob name**. `reader.py` (Container B) polls the container with `list_blobs()` — up to 12 attempts at 10-second intervals — and reports any token names it observes.

The choice to carry the signal in the *namespace* rather than in blob contents is deliberate, and it is the part of the design we would most want a standards body to notice. Content-level controls — scanning, encryption at rest, DLP on object bodies — do not see this channel at all. The blob is empty. Everything transmitted is transmitted by the act of creating a name that the other party can enumerate. This mirrors the incident's mechanism, where a cache namespace served as the message board, and it generalizes to any shared resource whose key space is writable and listable: cache keys, queue names, tag values, lock files, artifact coordinates.

The containment control

We designed two variants and implemented one. Both are **additive modules** activated by a single parameter (`containmentMode` , default `none`), so the base infrastructure is never rewritten and the "before" configuration remains exactly reproducible.

Variant (a) — per-run namespace isolation (*designed, not deployed*). Each sandbox receives its own per-run container; no shared rendezvous point exists. Minimal complexity, directly neutralizes the incident's mechanism. Its limitation is that both sandboxes still hold account-wide credentials, so it defends against accidental rendezvous but not against an adversarial agent that enumerates.

Variant (b) — short-lived, least-privilege, per-sandbox scoped SAS (*implemented and deployed*). We selected this variant because it attacks the root cause rather than the symptom, and because it admits a strictly stronger form of third-party verification. `containment_sas.bicep` mints two Shared Access Signatures with **disjoint scope and disjoint permissions**:

	Writer (<code>sandbox-a</code>)	Reader (<code>sandbox-b</code>)
<code>canonicalizedResource</code>	<code>/blob/<account>/wr-<runId></code>	<code>/blob/<account>/rd-<runId></code>
<code>signedPermission</code>	<code>cw</code> (create + write)	<code>rl</code> (read + list)
<code>signedResource</code>	<code>c</code> (container)	<code>c</code> (container)
TTL	30 minutes	30 minutes

Critically, in this mode **the storage account key is never passed to either sandbox**. The `sandbox.bicep` module supplies either the account key *or* a scoped SAS, never both, and the credential the sandbox receives is the only credential it has. The writer cannot list or read — not even its own container. The reader cannot write, and cannot reach the writer's container at all.

The full testbed is seven files: three base templates, one containment module, two agent scripts, one deploy script.

How we tested it

Both runs used an identical image (`mcr.microsoft.com/devcontainers/python:3.12`), identical container specs (1 vCPU, 1 GB), identical scripts, and the same storage account — a controlled comparison in which the containment mode is the only manipulated variable. Runs are separated by a cost-traceability tag (`environment=before-control` / `after-control`) applied at both resource-group and individual-resource level.

Before run — `containmentMode=none` . Both sandboxes receive the account key and are pointed at the same container, `covert-channel` .

```
[WRITER] Writing token as a zero-byte blob NAME: COVERT_CHANNEL_TEST_1789250624
[WRITER] Token written. blob_name = COVERT_CHANNEL_TEST_1789250624
[READER] Attempt 1/12: no tokens yet.
[READER] Attempt 2/12: no tokens yet.
[READER] RECEIVED token via shared resource: COVERT_CHANNEL_TEST_1789250624
[READER] Channel CONFIRMED OPEN. Tokens received: 1
```

The writer wrote at 22:03:44 UTC; the reader observed the token at 22:03:52 UTC, on its third poll — **8 seconds** from write to receipt, between two containers with no network route to each other.

After run — `containmentMode=scoped-sas`. Each sandbox receives only its own scoped SAS and its own container.

```
[WRITER] Credential: scoped SAS URL (least-privilege, write-only container).
[WRITER] Token written. blob_name = COVERT_CHANNEL_TEST_1789252005
[READER] Credential: scoped SAS URL (read/list on its OWN container only).
[READER] Attempt 1/12 ... Attempt 12/12: no tokens yet.
[READER] No tokens received in window. Channel appears CLOSED.
```

The writer *succeeded* — this matters, because it rules out the trivial failure mode in which the channel appears closed only because the sender broke. The token was written; it was written into a namespace the reader has no access to. The reader then exhausted its full 12-attempt window (~125 s, versus ~26 s in the before run where it exited early on receipt). The longer runtime is itself corroborating: this is a completed negative observation, not a truncated test.

Independent verification: two credential probes

Log output alone is weak evidence — it asks the reviewer to trust our harness. So we tested the *credentials themselves*, which is evidence a third party can regenerate without any access to our lab. Both probes return HTTP 403, and the fact that they fail by **two different mechanisms** is the substantive result:

Probe 1 — cross-container. The reader's SAS (`r1` , scoped to `rd-<runId>`) is used to LIST the writer's container:

```
HTTP 403 · <Code>AuthenticationFailed</Code>
"Signature did not match. String to sign used was r1 ...
/blob/ccstor3xnygvmi7f3z/wr-6a50151453977"
```

The signature is cryptographically bound to the canonicalized resource path of the reader's *own* container. Presenting it against a different container invalidates the signature outright — the request is rejected **before authorization is evaluated**. This is a stronger property than "lacks permission": the credential is bound to its resource by construction, so there is no permission state in which it could succeed.

Probe 2 — same container, wrong permission. The writer's SAS (`cw` , scoped to `wr-<runId>`) is used to LIST its own container:

```
HTTP 403 · <Code>AuthorizationPermissionMismatch</Code>
"This request is not authorized to perform this operation using this permission."
```

Here the signature is valid — the credential authenticates correctly against its own container — and the request is denied at the authorization layer because `cw` excludes read and list. This is the least-privilege proof: even within its own scope, the writer's credential cannot be turned into a receive channel.

Together the probes close both directions. Neither sandbox, alone or in collusion, holds an operation that would let it observe a token placed by the other.

Cost

Run	Writer	Reader	Resource-seconds	Compute cost
before	7.836 s	25.882 s	33.718	\$0.00042
after	7.904 s	125.445 s	133.349	\$0.00167
			Total	\$0.00209

Computed from container start/finish timestamps against published East US Standard Linux ACI rates (\$0.0405/vCPU-hour, \$0.00445/GB-hour; Azure Retail Prices API, retrieved 2026-09-12). The $\sim 4\times$ difference is fully explained by the reader running its complete poll window in the after run instead of exiting early on receipt — not by any infrastructure difference. Storage cost is negligible (zero-byte blobs, minutes of existence).

We report cost because it bears on whether this can be a *mandated* test. At roughly one fifth of a US cent per before/after cycle and under four minutes wall-clock, cost is not a reason to omit verification.

Main finding

Declared isolation is not verified isolation. A containment standard that does not mandate an executable covert-channel test against shared mutable resources cannot distinguish between the two. This testbed is that test.

The before run establishes the first half. Two sandboxes satisfying a defensible, template-auditable isolation specification — separate resource groups, no shared VNet, no inbound exposure, no anonymous storage access — communicated in 8 seconds. Every element of the declared isolation was true. None of it was sufficient, because none of it addressed the resource they shared. An auditor reading that template would have found nothing wrong, and would have been right about everything the template said.

The after run establishes the second half: the gap is closeable, cheaply, and closure is demonstrable to someone outside the lab. The control is not exotic — scoped, short-lived, least-privilege credentials are standard practice. What was missing in the incident was not the technique but the **test that**

would have revealed the technique was needed.

Three properties, we think, are what make this a candidate for a standard rather than a one-off experiment:

1. **It is executable, not declarative.** The output is an observed event (`CONFIRMED OPEN`) or an observed non-event (`CLOSED`) over a bounded window, not a statement about architecture.
2. **It is third-party verifiable without internal access.** The credential probes are reproducible by any reviewer who can mint SAS tokens with the declared scope. A reviewer does not have to trust our logs, our harness, or our word. A `200` where we report `403` falsifies the claim.
3. **It is negligible to run.** \$0.0021 and under four minutes, which removes cost as an argument against requiring it per-configuration rather than once-per-architecture.

The practical recommendation we would put to a containment standard: for every resource that two nominally-isolated agents can both reach, require a demonstrated attempt to pass a token through it — carried in the resource's *namespace*, not its contents — with the result recorded as evidence, and with the control's effect verifiable by an external reviewer. Isolation that has not been attacked in this narrow, benign way should be described as *declared*, and not as *verified*.

Limitations and dual-use considerations

We are explicit about what this testbed does **not** demonstrate. The gap between our claim and a general isolation guarantee is large, and we would rather state it than have it discovered.

It does not reproduce the real attack vector. No Jinja2 template injection, no HDF5 deserialization exploit, no privilege escalation, no registry cache-proxy zero-day. We pass a benign token between two cooperating processes. This was a deliberate design constraint, not a limitation we ran out of time to fix: the research question is whether a channel *exists* through a shared resource, and that question is answerable without weaponizing anything. A testbed that reproduced the escape would be substantially more dangerous to publish and would not answer the question any better.

It does not test isolation against a host- or kernel-level adversary. Our scope is channels through shared *storage* resources. An adversary with container-escape capability, hypervisor access, or kernel-level primitives is entirely outside what we measured, and nothing here should be read as evidence about side channels in shared CPU, memory, or scheduling. Real sandbox escapes — including the one that initiated the incident — often begin at exactly those layers. Our result is about what happens *after* isolation is nominally intact, not about whether it can be broken from below.

The SAS "single-use" property is approximated, not implemented. Azure Shared Access Signatures are bearer tokens valid until expiry; they are not natively single-use. What we implemented is a short TTL (30 minutes), disjoint per-sandbox container scope, and non-overlapping minimal permission sets. A stored access policy with explicit revocation would provide true one-time semantics; we deliberately left it out to keep scope minimal for the sprint deadline. The accurate description of this control is **"short-lived, least-privilege, per-sandbox scoped SAS"** — it should not be characterized as single-use, and we have avoided that phrasing throughout.

Scale. We tested with 2 sandboxes. The incident involved ~1,200, of which ~700 participated. A pairwise channel test does not straightforwardly extend to an n-party rendezvous problem: with 1,200 agents, the relevant question includes discovery dynamics, partial connectivity, and whether a control that holds pairwise degrades under concurrent load and contention on the shared resource. We make no claim that our result scales, and we regard this as the most important open question, not a minor caveat.

Single cloud, single resource type. One provider (Azure), one shared resource (blob storage). The namespace-as-channel pattern should generalize to caches, queues, artifact registries, and tag stores, but we have not demonstrated that.

The negative result is bounded by the observation window. `CLOSED` means no token was observed in 12 attempts over ~125 seconds. It is not a proof of impossibility. A channel with latency exceeding the window, or one requiring active probing the reader does not perform, would produce the same output. The credential probes are what raise this above a bounded-window negative — they establish that no permitted operation exists, not merely that none succeeded in time.

Dual-use

The technique demonstrated here — using a shared resource's namespace as a communication medium — is a genuine covert-channel primitive, and we should not pretend otherwise. We judge publication net-positive on the following grounds.

The primitive is neither novel nor difficult; covert channels through shared mutable state are long-established in the security literature, and the incident demonstrates that agents discover the pattern without instruction. The information asymmetry runs the wrong way: operators who need to *test* for this channel currently lack a cheap executable harness, while the capability to *use* it requires nothing we are adding. What we contribute is the test, the control, and the third-party verification method — the defensive half.

We have constrained the artifact accordingly. It passes a benign token with no payload; it contains no exploit, no escape primitive, and no privilege escalation; it is inert against any system other than a storage account the runner already controls. Someone intending harm gains nothing from this repository that they did not already have. Someone intending to verify containment gains a runnable test that costs a fifth of a cent.

The residual risk we take most seriously is misuse as false assurance: an operator runs this, observes `CLOSED`, and reports "isolation verified" in a sense far broader than what was measured. That failure mode would recreate the exact problem the testbed exists to expose — a declaration outrunning its evidence. This is why the limitations above are stated as prominently as the result, and why we would want any standard that adopts this test to adopt its scope statement with it.

What's next

Scale to n-party. The highest-value extension, and the one that most directly addresses the gap between our evidence and the incident. Move from 2 sandboxes to tens and then hundreds, and measure whether pairwise-verified controls hold under concurrent contention, and how rendezvous dynamics change when many agents share one namespace.

Generalize the resource class. Apply the same namespace-token test to package registry caches (the incident's actual medium), message queues, artifact coordinates, and tag stores — across more than one cloud provider.

Implement true revocation. Add stored access policies so the single-use claim can be made honestly rather than approximated, and measure the operational cost of per-run credential revocation at scale.

Harden the negative result. Extend beyond a bounded poll window toward a stronger statement: enumerate the operations each credential permits and show that none constitutes a channel. The credential probes are a first step; a systematic capability enumeration would be a real one.

Package as a conformance test. The natural product is a provider-agnostic harness that takes a declared isolation boundary and a list of shared resources, and emits a signed before/after evidence bundle that an external auditor can re-verify. That is the form in which this could become a requirement rather than a demonstration.

Close variant (a) as a comparison arm. Deploy the namespace-isolation variant and compare its failure modes against the credential-scoping variant under an adversarial reader that actively enumerates, to quantify the difference we currently argue for on design grounds alone.

Evidence index

All artifacts are in the accompanying repository.

Path	Contents
evidence/before/	Writer/reader logs, container timings, resource tags, run summary — channel <code>CONFIRMED OPEN</code>
evidence/after/	Same structure, plus <code>credential_probe.txt</code> — channel <code>CLOSED</code>
evidence/after/credential_probe.txt	Both 403 probes with full request/response and interpretation
main.bicep , storage.bicep , sandbox.bicep	Base infrastructure; isolation properties auditable here
containment_sas.bicep	Containment control, variant (b)
writer.py , reader.py	Agent scripts (benign token only)
deploy.sh	<code>./deploy.sh <SUB_ID> <before-control\ after-control></code>

Reproduction. `./deploy.sh <SUBSCRIPTION_ID> before-control` , then the same with `after-control` and `containmentMode=scoped-sas` . Total cost ~\$0.0021.

Sources

[1] Larcher, H., Carreira, A., et al. "Agent Intrusion: Technical Timeline." *Hugging Face Blog*, 2026-07-27. <https://huggingface.co/blog/agent-intrusion-technical-timeline>

[2] Greenblatt, R., Cotra, A., Wijk, H. "OpenAI–Hugging Face Incident Investigation." *METR*, 2026-08-26. <https://metr.org/blog/2026-08-26-openai-hugging-face-incident-investigation/>

Both sources were fetched and verified against the citations above at the time of writing (2026-09-12). None of this report's experimental results depend on these citations for validity — they establish the motivating context only, and the covert-channel mechanism itself ([2], §"Agent Discovery and Communication Mechanism") is independently reproduced, in benign form, by the testbed's before/after runs.